

Cross CLI Compatibility Routing

A universal migration framework for Claude Code, Codex, and Gemini CLI. Build one route map, keep the adapters thin, and keep the public surface aligned.

What This Guide Covers

This guide is for repos that need to move between coding shells without rewriting their operating model. The goal is not feature parity. The goal is a stable routing contract with explicit fallback behavior.

PDF Highlights

- A neutral route map that survives CLI swaps.
- What belongs in a universal layer versus an adapter layer.
- A migration checklist for Claude Code, Codex, and Gemini CLI.
- A copy-ready prompt you can paste into the active CLI.

Universal Routing Rules

- Keep one routing vocabulary: direct answer, research, planning, execution, verification.
- Move shared operating rules into neutral markdown files.
- Put provider-specific commands behind clearly labeled adapter notes.
- Define the fallback when a tool or connector does not exist in the current CLI.
- Update public docs, sitemap references, and machine-readable docs when routes change.

Migration Sequence

- 1 Inventory assumptions: file names, model names, slash commands, wrappers, and startup conventions.
- 2 Extract the universal layer: keep the workflow shared and the CLI adapters small.
- 3 Normalize route behavior: define goals, preferred tools, and fallback paths for each route type.
- 4 Verify the public surface: page, PDF, sitemap, and llms-style references should all agree.

Compatibility Notes

Claude Code: richest native agent workflow, but do not let Claude-only file names leak into the universal layer.

Codex: strong execution shell, but avoid assuming Claude-style skills or slash commands exist.

Gemini CLI: works best when the route map is already documented cleanly and fallbacks are explicit.

Universal Migration Prompt

You are migrating this workspace so the same operating model works in Claude Code, Codex, and Gemini CLI.

Objective:

Create a cross-CLI compatibility routing layer. Keep one routing model, one verification standard, and one set of sh

Fill these in before you start:

- Current CLI: [CLAUDE CODE / CODEX / GEMINI CLI]
- Secondary target CLI(s): [CLAUDE CODE / CODEX / GEMINI CLI]
- Repo root: [PATH]
- Current instruction files: [CLAUDE.md, AGENTS.md, GEMINI.md, README sections, etc.]
- Current routing docs: [ROUTING.md, INDEX.md, CONTEXT.md, TASKS.md, none]
- Package manager: [npm / pnpm / yarn / bun]

Work in this order:

1. Inventory CLI-specific assumptions.
2. Separate universal rules from adapter notes.
3. Normalize routing.
4. Replace provider-specific references.
5. Add compatibility notes.
6. Verify.

Required output:

- A short migration summary
- Files changed and why
- Remaining CLI-specific gaps
- The final route map
- Verification results

Constraints:

- Do not expand scope beyond compatibility routing.
- Prefer localized edits over a broad rewrite.
- Do not invent tools that do not exist in the active CLI.
- If a tool is missing, define the fallback path explicitly.
- Preserve behavior first, then improve wording second.

Workshop page: workshop.mastermindshq.business/giveaways/cross-cli-compatibility-routing